AI PRINCIPLES & TECHNIQUES

PROGRAMMING ASSIGNMENT 3

Radboud University

29 November 2024

Levente Bódi s1122062

Konstantinos Konstantinou s1118596

Introduction

Variable Elimination is fundamental to artificial intelligence, and it is a simple and general algorithm in probabilistic models, such as Bayesian Networks which will also be one focus of this paper. According to the basic probability distribution tables the algorithm calculates the probability distribution of the query variable taking observed variables into consideration as well.

In this report we focus on the implementation of a general variable elimination algorithm, which is used to evaluate Bayesian Networks wrote in .bif format.

It is also worth noting that Variable Elimination is an NP-hard problem, so there exist heuristics which one may follow to optimize performance. First, we utilized only the order of the nodes, then we implemented contained-in-fewest-factors-first heuristics.

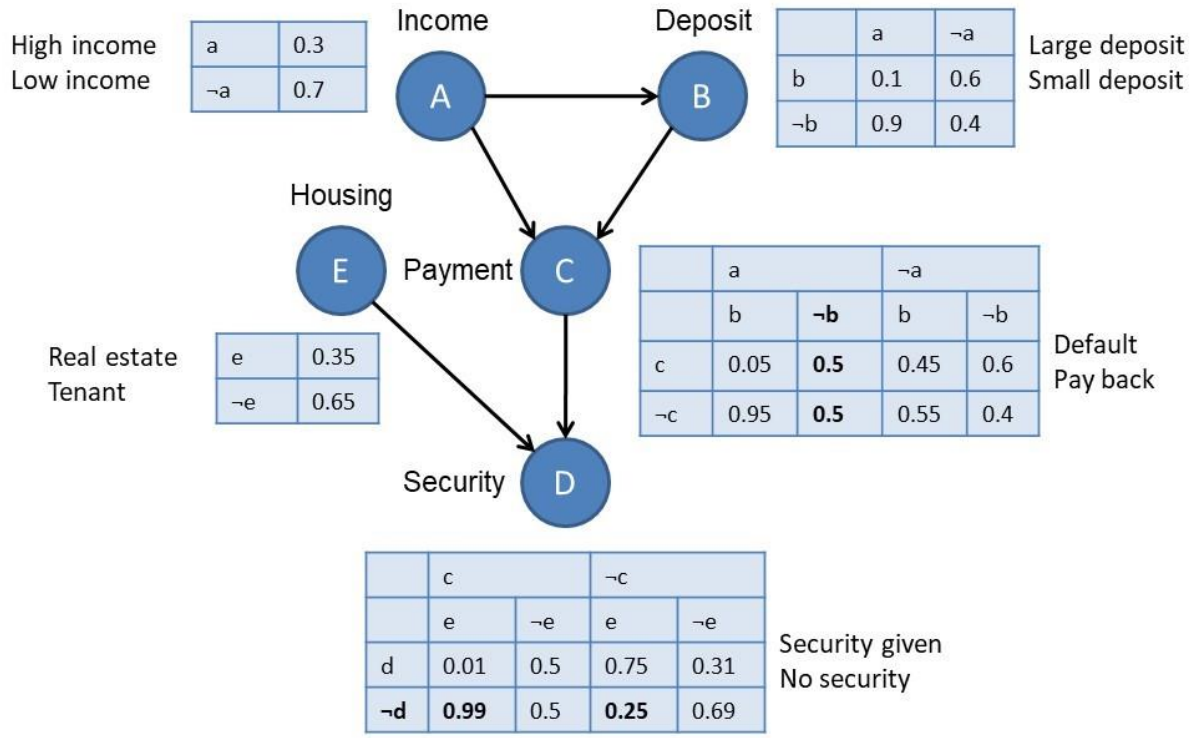
Method

Bayesian Networks

To be able to understand the Variable Elimination algorithm we would like to clarify and instantiate what Bayesian Networks are.

A Bayesian Network is a directed acyclic graph, where the directed edges originate from the parent nodes and flow to their children. When conceptualizing such a graph the parents of node X_i come before X_i , so if X_j and X_k are parents of X_i the ordering looks as follows: (X_j, …,X_k, …, X_i, …, X_n, …).

An example of a Bayesian Network:



Bayesian Networks: Lessons Learned from the Past! | DataMiningApps

In this example variable Income, node A is the parent of both Deposit and Payment, namely nodes B and C and has 2 values, Low income with probability 0.7 and High income with probability 0.3. Deposit is both the child of Income and the parent of Payment.

On the figure the probability distribution tables are also indicated in form of tables.

The joint probability distribution of Bayesian Networks can be factorized:

$P(X_1, \ldots, X_n) = \Pi^n_{i=1} P(X_i \mid X_1, \ldots, X_{i-1}) = \Pi^n_{i=1} P(X_i \mid \text{parents}(X_i))$

This will be really useful for the Variable Elimination algorithm.

Variable Elimination

As mentioned in the introduction the Variable Elimination algorithm is focusing on obtaining the probability distribution of the query variable with respect to all other probability distributions of other variables, while taking observed variables also into consideration.

Before beginning with the actual algorithm, one should take barren nodes into account, nodes which are neither evidence, nor target nodes, have no children, or have only barren nodes as children. These nodes do not have an influence on the outcome, so can be excluded from the computation.

By trying to determine the probability distribution of a certain variable we should take nodes into consideration which can pass information around and can have influence on the actual result.

Another approach can be to transform the graph into a tree-like structure, combining nodes into bags if needed.

A factor in the Variable Elimination algorithm is a 2D matrix which contains a real number of each place. In the abovementioned example a factor can be Income with A and ¬A, associated with marginal probabilities of 0.3 and 0.7

High income	a	0.3
Low income	¬a	0.7

The steps of the algorithm:

1. **Identify query and observed variables**
2. **Construct the production formula to compute the query**
3. **Reduce dependencies**
4. **Identify factors and reduce observed variables**
5. **Fix an elimination ordering**
6. **Multiply factors containing the variable next in the elimination order**
7. **Sum out the variable next in the elimination order to obtain a new factor**
8. **Remove the multiplied factors from the list and add the new factor to the list**
9. **Repeat steps 6-8 until only one factor remains with the query variable**
10. **Normalize the obtained result of the query variable to obtain a probability distribution**

In the following we will touch upon these steps shortly.

1. **Identify query and observed variables**

The query variable is the variable for which we want to have a result in the end, and the observed variables are the variables for which we have an observation. For example, an observation in the abovementioned example for variable Income could be that it is High, meaning that we should not take Low income into consideration when computing the result of our query variable.

2. **Construct the production formula to compute the query**

By constructing the production formula for the query, we sum all factor-product together with respect to the variables, which are not the query. So, if D is our query variable from our example the product formula will look as follows:

$P(D) = \Sigma_{A, B, C, E}\ P(A)*P(B|A)*P(C|A,B)*P(D|A,B,C)*P(E|A,B,C,D)$

3. **Reduce dependencies**

Reducing the dependencies means that we remove the variables from the factors on which the variable is not dependent, thus, we eliminate variables form the equation where there are no arrows. In our example the equation will look as follows then:

$P(D) = \Sigma_{A, B, C, E}\ P(A)*P(B|A)*P(C|A,B)*P(D|E,C)*P(E)$

4. **Identify factors and reduce observed variables**

By identifying factors, we construct a 2D matrix which contains a real number of each place as probabilities. If we work with observed variables, we eliminate the other values of the observed variable. For example, if we know that Income is High, we will eliminate the other parts of Income, namely Income being Low, since these are not needed in the computation further:

High income	a	0.3
~~Low income~~	¬a	0.7

5. **Fix an elimination ordering**

The elimination order of the variables can be random, or consecutive in the order of the variables, however there are some orders with heuristics which are more efficient than others, meaning that following these require less calculations. For example, such heuristics can be least-incoming-arcs first or contained-in-fewest-factors-first heuristics.

In our implementation we implemented and experimented with the latter, namely contained-in-fewest-factors-first heuristics, which takes all variables into consideration and starts with the evaluation which are present in the least factors, sparing some calculations.

6. **Multiply factors containing the variable next in the elimination order**

If a variable is present in more than one factors the factors should be multiplied together. This means that for each factor the values should be aligned, and these should be merged according where they have the same value by multiplying their probabilities. So in our example, if we have A next in the elimination order and P(A)* P(C|A,B) and a is 0.3, and a,b,c is 0.05, these should be multiplied together to obtain the new row a,b,c in the new factor P(A,B,C) from which A can be summed out in the next step. This should be done with all rows to actually obtain P(A, B,C).

7. **Sum out the variable next in the elimination order to obtain a new factor**

In order to actually reduce the number of variables, the variable next in the elimination order should be summed out from the factor. This is done via considering all other states of the variables in the factor where the variable next in the elimination order is and summing the probabilities together, irrespective of the values of the variable next in the elimination order.

	a		¬a	
	b	¬b	b	¬b
c	0.05	**0.5**	0.45	0.6
¬c	0.95	**0.5**	0.55	0.4

Default
Pay back

So if we were to sum out A in this case, the row for b,c would be 0.05+0.45. The whole table is as follows:

B	C	val
b	c	0.05+0.45 = 0.455
b	¬c	0.95+0.55 = 1.5
¬b	c	0.5+0.6 = 1.1
¬b	¬c	0.5+0.4 = 0.9

8. **Remove the multiplied factors from the list and add the new factor to the list**

When finalizing steps 6 and 7 we will obtain a new factor, which should be added to the list of factors. The factors which were multiplied together should be removed from the list.

9. **Repeat steps 6-8 until only one factor remains with the query variable**

Steps 6-8 should be repeated until the only factor remains with the query variable. An example results of an example factor $f_{10}(D)$ could be

D	val
d	0.71
¬d	0.12

10. **Normalize the obtained result of the query variable to obtain a probability distribution**

In the previous step it is obvious that 0.71 and 0.12 do not add up to 1, so it cannot be considered as a probability distribution in itself. To be able to obtain a real probability distribution the result should be normalized. This can be done via the formula:

$f_{10}(D)/ \Sigma_E (f_{10}(D))$. This means that 0.71 and 0.12 should both be divided by 0.71+0.12, from which the probability distribution is as follows:

D	val
d	0.855
¬d	0.145

Results

We experimented with contained-in-fewest-factors-first heuristics and we implemented the code in such a way that it is robust to variables with multiple values if given in .bif format.

From the log file it became clear that if variables with multiple values are integrated, then the number of computations grow, since the algorithm need to calculate more values when repeating steps 6 and 7.

We tested out small examples of Bayesian Networks, so the time difference between factors with Boolean values and multiple values was negligible but with large networks it is recommended to be careful, since increasing the number of variables and increasing the number of values is exponential.

We also created a log file, where it is listed what the algorithm exactly computes and how it does it. It starts with listing the initial factors and the query variables. It gives the elimination order after, and it lists the factors after eliminating each variable, with the final, normalized factor in the end.

Discussion

A major limitation of a Variable Elimination algorithm is that it is NP-hard, which arises from the nature of Bayesian Networks. As the number of variables increases, the joint probability distribution becomes exponentially larger, which makes it difficult to manage and compute. In Bayesian Networks, performing inference can be computationally intensive, especially for networks with many variables or intricate dependencies. Exact inference methods may become unfeasible, leading to the need for approximate inference techniques.

Conclusion

In this report, we have explored the Variable Elimination algorithm as a fundamental technique for performing inference in Bayesian Networks. The algorithm efficiently calculates the probability distribution of a query variable by systematically eliminating other variables while reducing computational complexity. We have discussed the steps involved in the Variable Elimination process, including the identification of query and observed variables, the construction of the production formula, the elimination of variables, and the final normalization of the result.

Additionally, we have highlighted the NP-hard nature of the Variable Elimination problem, which becomes increasingly challenging as the number of variables and the complexity of the network grow. To mitigate this, we implemented and tested heuristics such as the contained-in-fewest-factors-first heuristic, which optimizes the elimination order to improve performance. The experimentation showed that while the algorithm performs well on small Bayesian Networks, its computational cost can grow rapidly with larger networks or when

dealing with variables that have multiple values. This exponential growth underscores the importance of considering approximate inference methods for large-scale networks.

Despite its limitations, the Variable Elimination algorithm remains a powerful tool for probabilistic inference in Bayesian Networks. The results of this study demonstrate its practical applicability and the potential for optimization through heuristic techniques. Future work could focus on further refining these heuristics, exploring alternative approximation methods, and scaling the algorithm to handle even larger and more complex networks.